Lab Code

Aditya Kumar
24TEC2CS512

Date 06/04/2026
Page 21

Assignment-03

Q1. Define deadlock. Explain the four necessary conditions for deadlock occurence. Compare deadlock prevention, avoidance, and detection techniques.

Ans⇒ A deadlock is a situation in an operating system where a group of processes are permanently blocked because each process is waiting for a resource that is held by another process in the group.

- Four Necessary conditions for Deadlock.

1. Mutual Exclusion.
   - At least one resource cannot be shared.
   - Only one process can use it at a time.
   Eg:- Printer, file, or hardware device.

2. Hold and wait.
   - A process is holding one resource and waiting for others.
   Ex:- A process holding a printer and waiting for a scanner.

3. No Preemption.
   - Resources cannot be forcibly taken away.
   - They must be ~~rest~~ released voluntarily by the process.

- Allow preemption
- Prevent circular wait

Advantage: NO deadlock possible
Disadvantage: Poor resource utilization

2. Deadlock Avoidance.
- System checks before allocation to ensure it remains in a safe state.
- Uses algorithms like Banker's Algorithm.

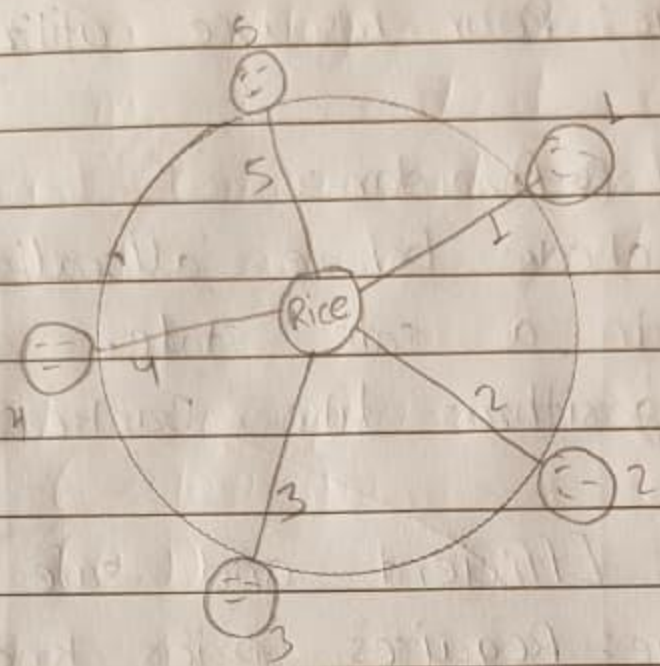
Advantage: Efficient and safe
Disadvantage: Requires prior knowledge of resource needs.

3. Deadlock Detection and Recovery.
- Deadlock is allowed to occur
- System detects it and takes action:
  - Terminate processes
  - Preempt resources

Advantage: Better resource utilization
Disadvantage: Overhed of detection and recovery.

Q2. Describe the Dining Philosopher problem with proper solution with a suitable example.

Ans⇒ The Dining Philosopher Problem is a classic synchronization problem in operating systems that illustrates issues like deadlock and resource sharing.



1. Thinking
2. Eating
3. No philosopher will starve
4. Philosopher may pick up only one chopstick at a
5. One philosopher can't pick up a chopstick that is already in the hand of a neighbour
6. From time to time, a philosopher gets hungry & tries to pick up the two (2) chopsticks that are closest to the philosopher.
7. When a philosopher thinks, she does not interact with colleagues.

Mutual Exclusion.

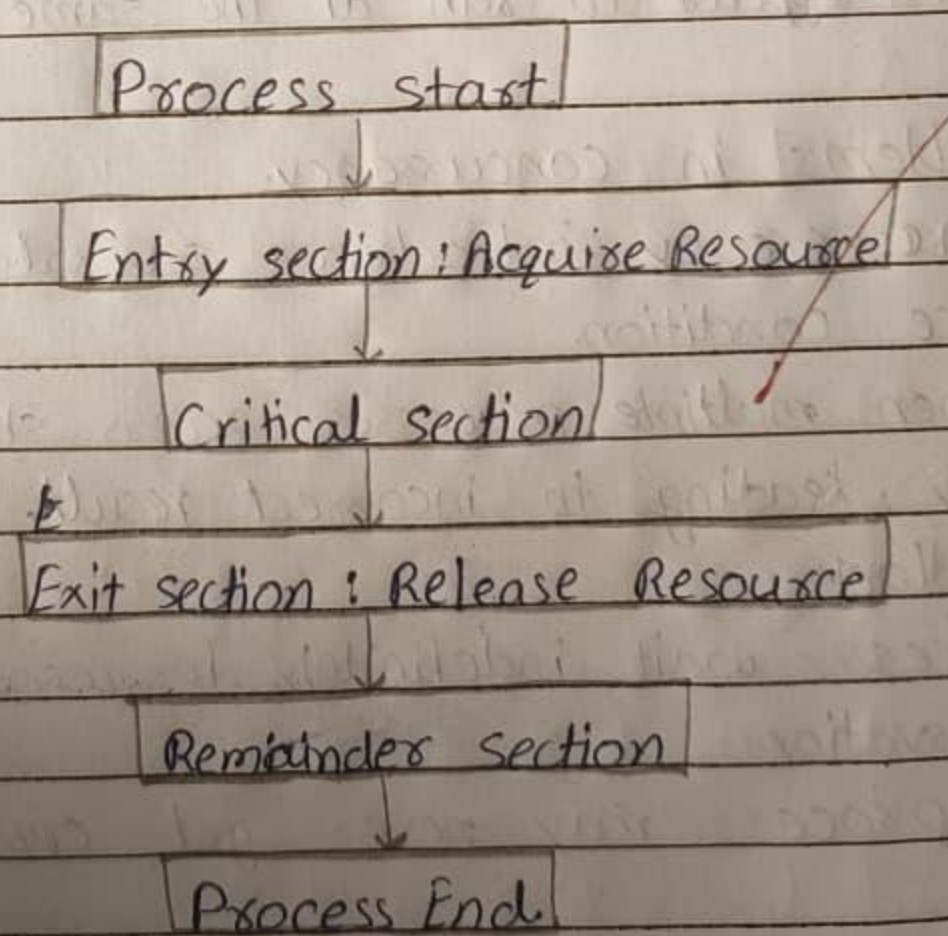
Mutual exclusion is a technique to ensure that only one process at a time can access a shared resource or critical section.

Critical section concept.

Each process has:

1. Entry Section → Request access
2. Critical Section → Access shared resource
3. Exit section → Release resource.
4. Remainder section → Other work.

Visualization



Requirements of Mutual Exclusion.

1. Mutual Exclusion
   - Only one process in critical section at a time.
2. Progress
   - If no process is in critical section, one should be allowed to enter.
3. Bounded Waiting
   - Every process gets a fair chance (no starvation).

Techniques to ~~Arrive~~ Achieve Mutual Exclusion.

- Locks / Mutex
- Semaphores
- Monitors
- Peterson's Algorithm.

Example.

If two processes try to update a bank balance simultaneously.

- Without mutual exclusion → incorrect balance
- With mutual exclusion → One process updates at a time → correct result.

Checked By
9/4/26

```
else
{
Block this process and place in the
  suspended list sleep ();
  }
}
```

Exit code :-

```
  up (semaphore s)
  {
  if (suspended list is empty)
  {
   s.value = 1;
   }
   else
   {
   select a process from suspended list
   and wake up ();
   }
}
```

Q 4. Explain the principle of Concurrency. Also discuss Mutual Exclusion.

Ans ⇒ Principle of concurrency.
Concurrency refers to the execution of multiple processes simultaneously or overlapping in time within a system.

8. Philosopher either can eat or think.
9. In the counter of the table, is a bowl of rice & the table is said with five (5) single chopsticks.

solⁿ → (I) Philosopher must allowed to pick up chopsticks, if both left & right are available.
(II) 4 philosopher may sit.

```
void philosopher (void);
{
  while (true)
  {
   Thinking ();

wait (chopsticks [i]);
wait (chopsticks ([(i+1) % 5]);

 Eat ();
   signal (chopstick[1]);
   signal (chop stick ([i+1) % 5]);
```

Q3. Define Semaphore Briefly discuss the different types of semaphore with its various operations.

Ans⇒ It is a integer variable, which is used in a mutual exclusive manner by various concurrent comperative process in order to achive synchronization.

Types of semaphore.

1. Counting $(-\infty \text{ to } +\infty)$
2. Binary $(0, 1)$

Process various operations:-

Entery code:-
P() Down

Exit code:-
V()

1. Counting:-

~~Ent~~

Entry code:-

```
Down (semaphore s)
{
S value = S value -1;
if (Svalue < 0)
```

```
{
 Put process (PCB) in the suspended list sleep();
 {
 else
    return ()
   }
  }
}
```

Exit code :

```
    Up (semaphore s)
    {
    S value = S value +1;
    if (s value ≤ 0)
    {
    Select a process from suspended list & wake
      up ();
      }
    }
```

2. Binary

Entry code :-

```
    Down (semaphore s)
    {
    if (s value = = 1)
    {
     S value = 0;
    {
```

- In a single CPU system → processes take turns (time sharing)
- In multi-core systems → processes can truly run in parallel

Key features of Concurrency.
- Multiple process execute independently.
- Processes may interact or share resource.
- Improve CPU utilization and system performance.

Example.
- While one program is downloading a file, another program is playing music.
- Both appear to run at the same time.

Problems in concurrency.
Concurrency introduces several challenges:

1. Race condition.
   When multiple processes access shared data simultane-ously, leading to incorrect result.
2. Deadlock
   Process wait indefinitely for resources.
3. Starvation
   A process may never get CPU/resources.

4. Circular Wait
- A circular chain of processes exists
- Each process is waiting for a resource held by the next process.

Ex:- $P_1 \rightarrow$ waiting for P_2
$P_2 \rightarrow$ waiting for P_3
$P_3 \rightarrow$ waiting for P_1

* ~~Fouta~~ Comparison of Deadlock Handling Techniques.

SN	Feature	Prevention	Avoidance	Detection.
1.	Idea	Eliminate one of the 4 conditions	Avoid unsafe states	Allow deadlock then detect.
2.	Approach	Restrictive resource allocation	Careful allocation (e.g., Banker's Algorithm)	Periodic checking.
3.	Resource utilization.	LOW	Moderate	High.
4.	System knowledge Needed.	NO.	Yes (future needs)	NO
5.	Complexity	Simple	Complex	Moderate
6.	Recovery Needed	NO	NO	Yes.

1. Deadlock Prevention.
- Break at least one nessary condition:
- Remove mutual exclusion (if possible)
- Avoid hold and wait.